2026-10-24 MySQL复合查询

- 1. MySQL 复合查询（上）：多表查询与自连接
  - 1.1 什么是复合查询
    - 1.1.1 不再局限于一张表
    - 1.1.2 多表查询的常见场景
    - 1.1.3 核心思想：多表转单表
    - 1.1.4 三张示例表
    - 1.1.5 本篇的知识链路
- 2. 基本查询回顾
  - 2.1 回顾的目的
    - 2.1.1 先保证单表查询基础稳定
    - 2.1.2 标量子查询仍属于查询的一部分
  - 2.2 八个基本查询案例
    - 2.2.1 工资和岗位组合筛选
    - 2.2.2 按部门升序、工资降序排序
    - 2.2.3 使用表达式计算年薪
    - 2.2.4 工资最高的员工
    - 2.2.5 工资高于平均工资的员工
    - 2.2.6 每个部门的平均工资和最高工资
    - 2.2.7 平均工资低于 2000 的部门
    - 2.2.8 每种岗位的人数和平均工资
  - 2.3 单表查询对复合查询的帮助
    - 2.3.1 复杂查询可以分层拆解
    - 2.3.2 分组字段与输出字段要匹配
- 3. 多表查询
  - 3.1 笛卡尔积
    - 3.1.1 直接组合两张表
    - 3.1.2 为什么会出现大量冗余
    - 3.1.3 用连接条件保留正确组合
  - 3.2 多表查询的基本写法
    - 3.2.1 逗号连接加 WHERE
    - 3.2.2 同名字段必须标明表名
    - 3.2.3 表别名让 SQL 更清晰
  - 3.3 多表查询案例
    - 3.3.1 显示员工姓名、工资和部门名称
    - 3.3.2 只显示 10 号部门的员工

        - 3.3.3 显示员工姓名、工资和工资等级
    - 3.4 多表查询的分析步骤
        - 3.4.1 先找字段来源，再找连接字段
        - 3.4.2 组合后仍是逻辑单表
        - 3.4.3 多表查询的常见错误
- 4. 自连接
    - 4.1 自连接的含义
        - 4.1.1 同一张表承担两个角色
        - 4.1.2 为什么必须使用不同的别名
    - 4.2 使用子查询查找 FORD 的上级
        - 4.2.1 先确定员工与领导的字段关系
        - 4.2.2 子查询写法为什么能返回一行
    - 4.3 使用自连接查找 FORD 的上级
        - 4.3.1 别名版 SQL
        - 4.3.2 逐步理解执行过程
        - 4.3.3 使用表别名的原因
    - 4.4 两种解法的比较
        - 4.4.1 子查询写法
        - 4.4.2 自连接写法
        - 4.4.3 不要只记格式
- 5. 本篇方法总结
    - 5.1 查询类型对照
        - 5.1.1 单表、多表与自连接
        - 5.1.2 多表查询的两个阶段
    - 5.2 原笔记核心结论的完整表达
        - 5.2.1 多表组合后可以看作单表
        - 5.2.2 FROM 是逻辑处理流程的起点
        - 5.2.3 子查询与 GROUP BY 规则不能被遗漏
    - 5.3 高频易错点
        - 5.3.1 直接查询多表却忘记关联条件
        - 5.3.2 关联字段相似却没有限定来源
        - 5.3.3 把笛卡尔积当成最终结果
        - 5.3.4 自连接别名角色混乱
        - 5.3.5 把逻辑顺序当成固定物理顺序

1. MySQL 复合查询（上）：多表查询与自连接

1.1 什么是复合查询

1.1.1 不再局限于一张表

前面学习的基础查询主要针对一张表。实际开发中，一次查询可能同时涉及：

1. 多张相互关联的表。
2. 写在 `SELECT`、`WHERE` 或 `FROM` 中的子查询。
3. 多个查询结果之间的集合运算。
4. 将上述结构继续嵌套或组合。
   因此，“复合查询”不只等于“在一个 `SELECT` 中嵌套另一个 `SELECT`”。嵌套子查询只是复合查询的一部分，多表查询、自连接和合并查询也属于这一主题。

1.1.2 多表查询的常见场景

关系型数据库为了减少冗余，通常会把不同类别的数据拆到不同表中。例如：

- `EMP` 保存员工编号、姓名、岗位、工资和部门编号。
- `DEPT` 保存部门编号、部门名称和部门地址。
- `SALGRADE` 保存工资等级及每一等级的最低、最高工资。

如果需求是“显示员工姓名、工资和所在部门名称”，员工信息来自 `EMP`，部门名称来自 `DEPT`。只查询其中一张表就缺少另一半信息，因此需要把两张表组合起来查询。

1.1.3 核心思想：多表转单表

解决多表问题的主线可以概括为：

> 先把多张表整合成一张有意义的中间结果表，再在这张逻辑表上完成筛选、投影、分组和排序。

例如，员工表中只保存部门编号，而部门表能根据部门编号提供部门名称。两表按部门编号对应后，就能得到一张同时包含“员工信息 + 部门信息”的逻辑表。

原笔记中的表述是：

> 解决多表问题的本质是设法把多表问题转化为单表问题，这是多表查询的核心思想。

这句话强调的是分析角度，而不是要求每次都把结果真正写入一张物理表。中间结果可以只存在于本次查询的执行过程中。

1.1.4 三张示例表

本课使用经典的员工、部门和工资等级数据：

表	本课主要字段	作用
`EMP`	`empno`、`ename`、`job`、`mgr`、`hiredate`、`sal`、`comm`、`deptno`	员工信息
`DEPT`	`deptno`、`dname`、`loc`	部门信息
`SALGRADE`	`grade`、`losal`、`hisal`	工资等级

理解字段含义时要特别注意：

- `EMP.deptno` 和 `DEPT.deptno` 表示同一种部门编号，可用于建立两表的对应关系。
- `EMP.mgr` 保存该员工上级领导的 `empno`。
- `comm` 是奖金，可能为 `NULL`。
- `SALGRADE.losal` 和 `hisal` 分别表示某个工资等级的下限与上限。

1.1.5 本篇的知识链路

本篇按以下顺序展开：

1. 用基础单表查询回顾筛选、排序、聚合和标量子查询。
2. 通过笛卡尔积理解多表组合的来源。
3. 使用连接条件删除无意义组合，得到有业务含义的数据。
4. 使用表别名区分同名字段和自连接中的不同角色。
5. 用自连接解决“员工与自己的上级都在同一张表中”的问题。

---

2. 基本查询回顾

2.1 回顾的目的

2.1.1 先保证单表查询基础稳定

多表查询最终仍要完成字段筛选、排序和聚合。如果单表查询的条件优先级、聚合规则或空值处理不熟悉，表连接部分会更容易出错。

课件在进入多表查询前，先用 `EMP` 表复习以下能力：

- 使用逻辑运算符组合条件。
- 使用 `LIKE` 匹配姓名首字母。
- 使用 `ORDER BY` 进行单列或多列排序。
- 使用表达式和别名计算年薪。
- 使用标量子查询取得最大工资或平均工资。
- 使用 `GROUP BY`、`HAVING` 和聚合函数完成部门统计。

2.1.2 标量子查询仍属于查询的一部分

例如：

```
SELECT ename, job
FROM EMP
WHERE sal = (SELECT MAX(sal) FROM EMP);
```

内层 `SELECT` 只返回一个工资值，外层查询再把这个值用于 `WHERE` 比较。这种“嵌套 `SELECT`”是复合查询的一种，但本部分只把它作为回顾，完整分类将在下一部分展开。

2.2 八个基本查询案例

2.2.1 工资和岗位组合筛选

需求：查询工资高于 500，或岗位为 `MANAGER` 的雇员，同时姓名首字母必须是大写 `J`。

```
SELECT *
FROM EMP
WHERE (sal > 500 OR job = 'MANAGER')
  AND ename LIKE 'J%';
```

这里必须用括号明确组合关系：

```
(工资条件 OR 岗位条件) AND 姓名条件
```

如果没有括号，`AND` 会先于 `OR` 执行，筛选条件就不再是三者共同约束下的结果。

2.2.2 按部门升序、工资降序排序

```sql
SELECT *
FROM EMP
ORDER BY deptno ASC, sal DESC;
```

多列排序从左到右依次生效：

1. 先按 `deptno` 升序排列。
2. 同一部门内部，再按 `sal` 降序排列。

2.2.3 使用表达式计算年薪

```sql
SELECT
    ename,
    sal * 12 + IFNULL(comm, 0) AS 年薪
FROM EMP
ORDER BY 年薪 DESC;
```

`comm` 可能为 `NULL`。`NULL` 参与算术运算后，结果仍可能为 `NULL`，所以使用 `IFNULL(comm, 0)` 把空奖金当作 0 处理。

这里还体现了 MySQL 的一个便利：`ORDER BY` 可以使用 `SELECT` 中定义的别名 `年薪`。

2.2.4 工资最高的员工

```sql
SELECT ename, job
FROM EMP
WHERE sal = (SELECT MAX(sal) FROM EMP);
```

执行思路是：

1. 内层子查询求出全公司的最高工资。
2. 外层查询筛选工资等于最高工资的员工。

不能简单写成 `SELECT ename, MAX(sal)` 就认为会自动返回最高工资对应的姓名，因为聚合函数压缩了原始行，普通列和聚合值需要符合分组规则。

2.2.5 工资高于平均工资的员工

```
SELECT ename, sal
FROM EMP
WHERE sal > (SELECT AVG(sal) FROM EMP);
```

内层查询返回全公司的平均工资，外层查询保留高于该平均值的员工记录。这仍然属于“子查询为外层 `WHERE` 提供比较值”的用法。

2.2.6 每个部门的平均工资和最高工资

```
SELECT
    deptno,
    FORMAT(AVG(sal), 2) AS 平均工资,
    MAX(sal) AS 最高工资
FROM EMP
GROUP BY deptno;
```

`GROUP BY deptno` 把员工按部门分成若干组，聚合函数分别计算每组的结果。

`FORMAT(value, 2)` 主要改变显示格式，保留两位小数；它不改变原字段的数据类型。

2.2.7 平均工资低于 2000 的部门

```
SELECT
    deptno,
    AVG(sal) AS avg_sal
FROM EMP
GROUP BY deptno
HAVING avg_sal < 2000;
```

判断依据是“分组后的平均工资”，所以不能用分组前的 `WHERE` 来筛选该聚合结果。

MySQL 中 `HAVING` 可以使用 `SELECT` 定义的别名。若考虑 SQL 可移植性，也可以重复写聚合表达式：

```sql
SELECT
    deptno,
    AVG(sal) AS avg_sal
FROM EMP
GROUP BY deptno
HAVING AVG(sal) < 2000;
```

2.2.8 每种岗位的人数和平均工资

```sql
SELECT
    job,
    COUNT(*) AS 人数,
    FORMAT(AVG(sal), 2) AS 平均工资
FROM EMP
GROUP BY job;
```

`COUNT(*)` 统计每组记录数，`AVG(sal)` 统计每组平均工资。这里的 `job` 出现在 `GROUP BY` 中，因此可以稳定地作为分组标识输出。

2.3 单表查询对复合查询的帮助

2.3.1 复杂查询可以分层拆解

面对一条复合查询，可以先问：

1. 最终需要哪些字段？
2. 原始数据来自哪些表？
3. 表与表之间通过哪些字段对应？
4. 哪些条件在组合前筛选，哪些在组合后筛选？
5. 是否需要分组、排序或限制返回行数？

把大问题拆成这些层次后，多表、嵌套和聚合都可以分别处理，再组合成一条完整 SQL。

2.3.2 分组字段与输出字段要匹配

原笔记记录了课件强调的一条规则：使用 `GROUP BY` 时，不能在 `SELECT` 中随意输出未被分组、也未被聚合的列。

例如：

```
SELECT ename, deptno, AVG(sal)
FROM EMP
GROUP BY deptno;
```

每个部门包含多名员工，而 `ename` 并不由 `deptno` 唯一确定。在启用 `ONLY_FULL_GROUP_BY` 时，这类写法会报错。

应选择符合分组粒度的字段：

```
SELECT deptno, AVG(sal)
FROM EMP
GROUP BY deptno;
```

---

3. 多表查询

3.1 笛卡尔积

3.1.1 直接组合两张表

如果把两张表直接写在 `FROM` 中：

```
SELECT *
FROM EMP, DEPT;
```

结果并不是只把两张表上下拼接，而是让 `EMP` 的每一行与 `DEPT` 的每一行逐一配对。这种组合叫作**笛卡尔积**。

如果 `EMP` 有 `m` 行、`DEPT` 有 `n` 行，原始组合数量就是：

```
m × n
```

3.1.2 为什么会出现大量冗余

展示直接查询 `emp, dept` 的结果：

```
mysql> select * from emp,dept;
+--------+--------+-----------+------+---------------------+---------+---------+--------+--------+------------+----------+
| empno  | ename  | job       | mgr  | hiredate            | sal     | comm    | deptno | deptno | dname      | loc      |
+--------+--------+-----------+------+---------------------+---------+---------+--------+--------+------------+----------+
| 007369 | SMITH  | CLERK     | 7902 | 1980-12-17 00:00:00 |  800.00 |    NULL |     20 |     10 | ACCOUNTING | NEW YORK |
| 007369 | SMITH  | CLERK     | 7902 | 1980-12-17 00:00:00 |  800.00 |    NULL |     20 |     20 | RESEARCH   | DALLAS   |
| 007369 | SMITH  | CLERK     | 7902 | 1980-12-17 00:00:00 |  800.00 |    NULL |     20 |     30 | SALES      | CHICAGO  |
| 007369 | SMITH  | CLERK     | 7902 | 1980-12-17 00:00:00 |  800.00 |    NULL |     20 |     40 | OPERATIONS | BOSTON   |
| 007499 | ALLEN  | SALESMAN  | 7698 | 1981-02-20 00:00:00 | 1600.00 |  300.00 |     30 |     10 | ACCOUNTING | NEW YORK |
| 007499 | ALLEN  | SALESMAN  | 7698 | 1981-02-20 00:00:00 | 1600.00 |  300.00 |     30 |     20 | RESEARCH   | DALLAS   |
| 007499 | ALLEN  | SALESMAN  | 7698 | 1981-02-20 00:00:00 | 1600.00 |  300.00 |     30 |     30 | SALES      | CHICAGO  |
| 007499 | ALLEN  | SALESMAN  | 7698 | 1981-02-20 00:00:00 | 1600.00 |  300.00 |     30 |     40 | OPERATIONS | BOSTON   |
| 007521 | WARD   | SALESMAN  | 7698 | 1981-02-22 00:00:00 | 1250.00 |  500.00 |     30 |     10 | ACCOUNTING | NEW YORK |
| 007521 | WARD   | SALESMAN  | 7698 | 1981-02-22 00:00:00 | 1250.00 |  500.00 |     30 |     20 | RESEARCH   | DALLAS   |
| 007521 | WARD   | SALESMAN  | 7698 | 1981-02-22 00:00:00 | 1250.00 |  500.00 |     30 |     30 | SALES      | CHICAGO  |
| 007521 | WARD   | SALESMAN  | 7698 | 1981-02-22 00:00:00 | 1250.00 |  500.00 |     30 |     40 | OPERATIONS | BOSTON   |
| 007566 | JONES  | MANAGER   | 7839 | 1981-04-02 00:00:00 | 2975.00 |    NULL |     20 |     10 | ACCOUNTING | NEW YORK |
| 007566 | JONES  | MANAGER   | 7839 | 1981-04-02 00:00:00 | 2975.00 |    NULL |     20 |     20 | RESEARCH   | DALLAS   |
| 007566 | JONES  | MANAGER   | 7839 | 1981-04-02 00:00:00 | 2975.00 |    NULL |     20 |     30 | SALES      | CHICAGO  |
| 007566 | JONES  | MANAGER   | 7839 | 1981-04-02 00:00:00 | 2975.00 |    NULL |     20 |     40 | OPERATIONS | BOSTON   |
| 007654 | MARTIN | SALESMAN  | 7698 | 1981-09-28 00:00:00 | 1250.00 | 1400.00 |     30 |     10 | ACCOUNTING | NEW YORK |
| 007654 | MARTIN | SALESMAN  | 7698 | 1981-09-28 00:00:00 | 1250.00 | 1400.00 |     30 |     20 | RESEARCH   | DALLAS   |
| 007654 | MARTIN | SALESMAN  | 7698 | 1981-09-28 00:00:00 | 1250.00 | 1400.00 |     30 |     30 | SALES      | CHICAGO  |
| 007654 | MARTIN | SALESMAN  | 7698 | 1981-09-28 00:00:00 | 1250.00 | 1400.00 |     30 |     40 | OPERATIONS | BOSTON   |
| 007698 | BLAKE  | MANAGER   | 7839 | 1981-05-01 00:00:00 | 2850.00 |    NULL |     30 |     10 | ACCOUNTING | NEW YORK |
| 007698 | BLAKE  | MANAGER   | 7839 | 1981-05-01 00:00:00 | 2850.00 |    NULL |     30 |     20 | RESEARCH   | DALLAS   |
| 007698 | BLAKE  | MANAGER   | 7839 | 1981-05-01 00:00:00 | 2850.00 |    NULL |     30 |     30 | SALES      | CHICAGO  |
| 007698 | BLAKE  | MANAGER   | 7839 | 1981-05-01 00:00:00 | 2850.00 |    NULL |     30 |     40 | OPERATIONS | BOSTON   |
| 007782 | CLARK  | MANAGER   | 7839 | 1981-06-09 00:00:00 | 2450.00 |    NULL |     10 |     10 | ACCOUNTING | NEW YORK |
```

可以看到，同一位员工会被拿去和多个部门逐一组合。例如 SMITH 实际属于 20 号部门，但结果中还会出现“SMITH + 10 号部门”“SMITH + 30 号部门”等没有业务意义的组合。

笛卡尔积本身不是最终答案，它只是两表组合的完整来源集合。

3.1.3 用连接条件保留正确组合

只需要保留“员工表部门编号 = 部门表部门编号”的记录：

```sql
SELECT *
FROM EMP, DEPT
WHERE EMP.deptno = DEPT.deptno;
```

此时每名员工只与自己的真实部门配对：

```
mysql> select * from emp,dept where emp.deptno=dept.deptno;
+--------+--------+-----------+------+---------------------+---------+---------+--------+--------+------------+----------+
| empno  | ename  | job       | mgr  | hiredate            | sal     | comm    | deptno | deptno | dname      | loc      |
+--------+--------+-----------+------+---------------------+---------+---------+--------+--------+------------+----------+
| 007369 | SMITH  | CLERK     | 7902 | 1980-12-17 00:00:00 |  800.00 |    NULL |     20 |     20 | RESEARCH   | DALLAS   |
| 007499 | ALLEN  | SALESMAN  | 7698 | 1981-02-20 00:00:00 | 1600.00 |  300.00 |     30 |     30 | SALES      | CHICAGO  |
| 007521 | WARD   | SALESMAN  | 7698 | 1981-02-22 00:00:00 | 1250.00 |  500.00 |     30 |     30 | SALES      | CHICAGO  |
| 007566 | JONES  | MANAGER   | 7839 | 1981-04-02 00:00:00 | 2975.00 |    NULL |     20 |     20 | RESEARCH   | DALLAS   |
| 007654 | MARTIN | SALESMAN  | 7698 | 1981-09-28 00:00:00 | 1250.00 | 1400.00 |     30 |     30 | SALES      | CHICAGO  |
| 007698 | BLAKE  | MANAGER   | 7839 | 1981-05-01 00:00:00 | 2850.00 |    NULL |     30 |     30 | SALES      | CHICAGO  |
| 007782 | CLARK  | MANAGER   | 7839 | 1981-06-09 00:00:00 | 2450.00 |    NULL |     10 |     10 | ACCOUNTING | NEW YORK |
| 007788 | SCOTT  | ANALYST   | 7566 | 1987-04-19 00:00:00 | 3000.00 |    NULL |     20 |     20 | RESEARCH   | DALLAS   |
| 007839 | KING   | PRESIDENT | NULL | 1981-11-17 00:00:00 | 5000.00 |    NULL |     10 |     10 | ACCOUNTING | NEW YORK |
| 007844 | TURNER | SALESMAN  | 7698 | 1981-09-08 00:00:00 | 1500.00 |    0.00 |     30 |     30 | SALES      | CHICAGO  |
| 007876 | ADAMS  | CLERK     | 7788 | 1987-05-23 00:00:00 | 1100.00 |    NULL |     20 |     20 | RESEARCH   | DALLAS   |
| 007900 | JAMES  | CLERK     | 7698 | 1981-12-03 00:00:00 |  950.00 |    NULL |     30 |     30 | SALES      | CHICAGO  |
| 007902 | FORD   | ANALYST   | 7566 | 1981-12-03 00:00:00 | 3000.00 |    NULL |     20 |     20 | RESEARCH   | DALLAS   |
| 007934 | MILLER | CLERK     | 7782 | 1982-01-23 00:00:00 | 1300.00 |    NULL |     10 |     10 | ACCOUNTING | NEW YORK |
+--------+--------+-----------+------+---------------------+---------+---------+--------+--------+------------+----------+
14 rows in set (0.00 sec)
```

“外键和其他表的信息做拼接”，本质上就是：

1. 通过两个表中共有的关联字段找到对应记录。
2. 把来自不同表的字段拼接进同一行结果。

3.2 多表查询的基本写法

3.2.1 逗号连接加 WHERE

传统写法是：

```
SELECT 字段列表
FROM 表1, 表2
WHERE 表1.关联字段 = 表2.关联字段;
```

可以把它理解为两个阶段：

```
FROM 阶段：形成两表的笛卡尔积
WHERE 阶段：删除不符合连接条件的组合
```

这种写法在语义上属于内连接，即只保留两边能够匹配的记录。

3.2.2 同名字段必须标明表名

`EMP` 和 `DEPT` 都有 `deptno` 字段。只写：

```
WHERE deptno = deptno
```

会让数据库无法判断两边各自来自哪张表。

正确写法：

```
WHERE EMP.deptno = DEPT.deptno
```

字段前的表名不是装饰，而是在多表查询中消除字段来源歧义。

3.2.3 表别名让 SQL 更清晰

表名较长或需要重复使用同一张表时，可以设置别名：

```
SELECT e.ename, d.dname
FROM EMP AS e, DEPT AS d
WHERE e.deptno = d.deptno;
```

设置别名后，`e` 代表 `EMP`，`d` 代表 `DEPT`。字段写成 `e.deptno` 或 `d.deptno`，来源更清楚，SQL 也更短。

3.3 多表查询案例

3.3.1 显示员工姓名、工资和部门名称

需求中的数据分别来自 `EMP` 和 `DEPT`，因此必须联合查询：

```
SELECT
    EMP.ename,
    EMP.sal,
    DEPT.dname
FROM EMP, DEPT
WHERE EMP.deptno = DEPT.deptno;
```

分析过程：

1. `FROM` 先取得两张表的候选组合。
2. `WHERE EMP.deptno = DEPT.deptno` 只保留部门编号一致的组合。
3. `SELECT` 从保留下来的组合中取出姓名、工资和部门名称。

这里给出的 `WHERE` 是两表的连接条件。省略它时，就会退化成笛卡尔积，原先正确的员工与部门对应关系也会被大量无意义组合淹没。

3.3.2 只显示 10 号部门的员工

```
SELECT
    ename,
    sal,
    dname
FROM EMP, DEPT
WHERE EMP.deptno = DEPT.deptno
  AND DEPT.deptno = 10;
```

这条语句同时包含两种条件：

条件	作用
`EMP.deptno = DEPT.deptno`	连接两张表，决定哪些行能够对应
`DEPT.deptno = 10`	对已经连接的结果做业务筛选

连接条件与普通筛选条件在逻辑作用上不同。更清晰的显式 `JOIN ... ON ...` 写法将在后续表连接课程中学习，但这里的核心语义已经相同。

3.3.3 显示员工姓名、工资和工资等级

`EMP.sal` 需要与 `SALGRADE` 中的工资区间比较：

```
SELECT
    ename,
    sal,
    grade
FROM EMP, SALGRADE
WHERE EMP.sal BETWEEN losal AND hisal;
```

连接条件不是简单的等值比较，而是“员工工资落在某个工资等级的上下限之间”。

`BETWEEN losal AND hisal` 等价于：

```
EMP.sal >= losal AND EMP.sal <= hisal
```

选择 `SALGRADE` 后，员工就能找到自己工资所在的等级。

3.4 多表查询的分析步骤

3.4.1 先找字段来源，再找连接字段

可以按下面的顺序分析：

1. 从需求中列出要显示的字段。
2. 确认每个字段来自哪张表。
3. 找出两张表共有的关联字段。
4. 写出连接条件。
5. 再加入普通业务筛选条件。
6. 最后考虑分组、排序或聚合。

例如“员工姓名 + 部门名称”：

```
ename -> EMP
dname -> DEPT
关联字段 -> EMP.deptno = DEPT.deptno
```

分析清楚字段来源后，SQL 的主体结构就基本确定了。

3.4.2 组合后仍是逻辑单表

两张表按连接条件组合后，可以把结果暂时看成一张包含两表字段的逻辑表：

ename	sal	deptno	dname
SMITH	800.00	20	RESEARCH
ALLEN	1600.00	30	SALES

后续的 `WHERE`、`GROUP BY`、`ORDER BY` 都可以理解为在这张逻辑表上继续进行单表操作。这就是“多表转单表”分析方法的具体体现。

3.4.3 多表查询的常见错误

漏写连接条件

如果只写 `FROM EMP, DEPT` 而没有 `WHERE EMP.deptno = DEPT.deptno`，结果会包含笛卡尔积中的大量无意义组合。

把业务筛选误当成连接条件

例如只写 `WHERE DEPT.deptno = 10`，虽然没有直接形成完整笛卡尔积，但两张表之间仍缺少正确的员工与部门对应条件。

同名字段没有表名

`deptno` 同时存在于 `EMP` 和 `DEPT` 中。若不说明表名，会出现字段歧义。

认为行数等于员工数

一对多关系或漏写条件都会改变结果行数。查询结果的行数必须结合连接关系分析。

---

4. 自连接

4.1 自连接的含义

4.1.1 同一张表承担两个角色

自连接是指在查询中把同一张表连接起来使用。

`EMP` 中同时保存了：

- `empno`：员工自己的编号。
- `mgr`：该员工上级领导的编号。

由于领导和员工都属于员工，两者都存放在 `EMP` 中。查询“某位员工的上级”时，需要让同一个 `EMP` 表分别扮演两个角色：

```
leader 角色：提供上级编号和姓名
worker 角色：提供普通员工信息
```

4.1.2 为什么必须使用不同的别名

如果两次写 `EMP` 而不加别名，数据库无法区分当前字段来自“作为领导的 EMP”还是“作为员工的 EMP”。

因此自连接通常写成：

```
FROM EMP AS leader, EMP AS worker
```

`leader` 和 `worker` 指向同一张物理表，但在本次查询中代表两个不同的逻辑角色。

4.2 使用子查询查找 FORD 的上级

4.2.1 先确定员工与领导的字段关系

需求：显示员工 FORD 的上级领导编号和姓名。

对应关系是：

```
FORD.mgr = 领导的 empno
```

第一步可以从 `EMP` 中查出 FORD 的 `mgr`：

```
SELECT mgr
FROM EMP
WHERE ename = 'FORD';
```

内层查询的结果可以理解为 `7566`。再把这个编号作为外层查询的 `empno`：

```sql
SELECT empno, ename
FROM EMP
WHERE empno = (SELECT mgr FROM EMP WHERE ename = 'FORD');
```

4.2.2 子查询写法为什么能返回一行

外层比较使用的是 `=`，因此内层子查询应当返回一个值。在示例数据中，FORD 只有一位直接上级，所以 `SELECT mgr ...` 返回一行。

这里的子查询写在 `WHERE` 中，充当外层的比较值。不是所有子查询都只能这样使用，下一部分还会学习返回多行、多个字段以及写在 `FROM` 中的子查询。

4.3 使用自连接查找 FORD 的上级

4.3.1 别名版 SQL

```sql
SELECT
    leader.empno,
    leader.ename
FROM EMP AS leader, EMP AS worker
WHERE leader.empno = worker.mgr
  AND worker.ename = 'FORD';
```

4.3.2 逐步理解执行过程

可以把两个角色对应的条件写成：

条件	含义
`leader.empno = worker.mgr`	领导自己的编号，等于某位员工记录的上级编号
`worker.ename = 'FORD'`	当前员工角色限定为 FORD

组合后：

1. `worker` 角色筛选到 FORD。
2. FORD 的 `mgr` 字段提供上级编号。

3. `leader.empno` 与这个编号匹配。
4. 输出 `leader` 角色对应的编号和姓名。

FORD 的上级是 JONES，所以最终得到 JONES 的 `empno` 和 `ename`。

4.3.3 使用表别名的原因

课件注释强调：同一个 `emp` 表必须先起不同别名，才能区分笛卡尔积中的两种角色。

```
FROM emp leader, emp worker
```

这里的 `leader` 和 `worker` 不是两张不同的物理表，而是同一张表在逻辑上的两个副本。别名把“员工自己的信息”和“员工上级的信息”分开表达。

4.4 两种解法的比较

4.4.1 子查询写法

```
SELECT empno, ename
FROM EMP
WHERE empno = (SELECT mgr FROM EMP WHERE ename = 'FORD');
```

特点：

- 先求 FORD 的上级编号。
- 再把该编号作为外层筛选条件。
- 例子较短，适合只取一个中间值的简单场景。

4.4.2 自连接写法

```
SELECT leader.empno, leader.ename
FROM EMP AS leader, EMP AS worker
WHERE leader.empno = worker.mgr
  AND worker.ename = 'FORD';
```

特点：

- 直接把员工角色和领导角色放在同一个连接查询中。
- 能同时使用两个角色上的字段。
- 当查询需要输出员工本人和领导的更多信息时，自连接通常更自然。

4.4.3 不要只记格式

自连接的难点不在 `FROM emp a, emp b` 这串写法，而在于明确两个别名分别承担什么业务角色，以及两个角色通过哪两个字段对应。

分析自连接时应先写清楚：

```
别名 A 代表什么？
别名 B 代表什么？
A 的哪个字段对应 B 的哪个字段？
```

---

5. 本篇方法总结

5.1 查询类型对照

5.1.1 单表、多表与自连接

查询类型	数据来源	核心问题
单表查询	一张物理表	如何筛选、排序、聚合
多表查询	两张或多张不同的表	表与表如何对应
自连接	同一张表出现两次	同一张表的两个业务角色如何对应

5.1.2 多表查询的两个阶段

```
先产生候选组合
    -> 再按连接条件保留正确对应关系
    -> 最后完成业务筛选与字段输出
```

对于逗号连接写法，候选组合主要来自 `FROM` 中的笛卡尔积；连接条件和普通筛选条件通常都写在 `WHERE` 中。分析时仍应人为区分这两类条件。

5.2 原笔记核心结论的完整表达

5.2.1 多表组合后可以看作单表

两张表并不是任意拼接就有意义。应先形成候选组合，再通过连接条件保留真实对应记录。得到中间结果后，可以把它看作一张包含两表字段的逻辑表，再完成筛选和投影。

5.2.2 FROM 是逻辑处理流程的起点

查询的逻辑处理顺序通常从 `FROM` 开始：

```
FROM / JOIN
    -> WHERE
    -> GROUP BY
    -> HAVING
    -> SELECT
    -> ORDER BY
    -> LIMIT
```

“`FROM` 最先执行”强调的是数据来源先被确定？实际执行时，数据库优化器可能调整物理执行策略，因此更准确的理解是“逻辑处理顺序”，不能把这条顺序当成每一次实际读写操作的固定时间线。

5.2.3 子查询与 GROUP BY 规则不能被遗漏

- 写在 `WHERE` 中的子查询可以为外层比较提供值。
- 多表连接与子查询都可以为复杂需求构造中间数据。
- 使用 `GROUP BY` 时，非聚合输出列必须符合分组规则。
- MySQL 可以允许某些扩展写法，但写 SQL 时仍应以清晰、可移植、含义明确为准。

5.3 高频易错点

5.3.1 直接查询多表却忘记关联条件

```
SELECT *
FROM EMP, DEPT;
```

这条语句得到的是笛卡尔积，不等同于“员工及其部门信息”。必须补充两表的连接条件。

5.3.2 关联字段相似却没有限定来源

`EMP` 和 `DEPT` 都有 `deptno`，应写：

```
WHERE EMP.deptno = DEPT.deptno
```

不能只写没有来源限定的 `deptno`。

5.3.3 把笛卡尔积当成最终结果

笛卡尔积是组合的来源，不是有业务含义的最终结果。保留哪些组合，取决于连接条件。

5.3.4 自连接别名角色混乱

自连接必须明确哪一侧是领导、哪一侧是员工。仅仅给两个 `EMP` 起别名还不够，还要写清：

```
leader.empno = worker.mgr
```

5.3.5 把逻辑顺序当成固定物理顺序

`FROM`、`WHERE`、`GROUP BY`、`HAVING`、`SELECT`、`ORDER BY` 和 `LIMIT` 主要用于理解查询语义。数据库优化器可以在不改变结果的前提下改变具体执行方式。